
Camera



Capturing

Introduction

- ▶ Mobile apps sometimes need to capture photos or videos from a camera
- ▶ Attn:
 - ▶ A device may have none, one or multiple cameras
 - ▶ Many options and modes may be used



Camera Package (3rd Party)

- ▶ React Native CLI project is required!
- ▶ Installation
 - ▶ `npm install react-native-vision-camera`
- ▶ Permission declarations:
 - ▶ **Android (AndroidManifest.xml file):**
 - ▶ `<uses-permission android:name="android.permission.CAMERA" />`
 - ▶ **iOS (Info.plist file):**
 - ▶ `<key>NSCameraUsageDescription</key>
<string>$(PRODUCT_NAME) needs access to your Camera.</string>`
- ▶ Usage
 - ▶ `import { useCameraDevices, Camera } from 'react-native-vision-camera';`

Troubleshooting

- ▶ If the following error happens when starting the app on Android:

```
The Android Gradle plugin supports only kotlin-android-extensions Gradle plugin version 1.6.20 and higher.  
The following dependencies do not satisfy the required version:  
project ':react-native-vision-camera' -> org.jetbrains.kotlin:kotlin-gradle-plugin:1.5.30
```

- ▶ Note that the versions may be different
- ▶ Then:
 - ▶ Go to `android/gradle.properties`
 - ▶ Add the following line:
 - ▶

```
buildscript {  
    ext {  
        ...  
        kotlinVersion = "1.6.20"  
        ...  
    }  
}
```
 - ▶ The version value should match the required one in the error message

Checking Permission

- ▶ `const p = await Camera.getCameraPermissionStatus();`
- ▶ Returns permission status that can be:
 - ▶ `"authorized"`: Your app is authorized to use said permission. Continue with using the `<Camera>` view.
 - ▶ `"not-determined"`: Your app has not yet requested permission from the user. Continue by calling the request functions.
 - ▶ `"denied"`: Your app has already requested permissions from the user, but was explicitly denied. You cannot use the request functions again, but you can use the Linking API to redirect the user to the Settings App where he can manually grant the permission.
 - ▶ `"restricted"`: (iOS only) Your app cannot use the Camera because that functionality has been restricted, possibly due to active restrictions such as parental controls being in place.

Requesting Permission

- ▶ `const p = await Camera.requestCameraPermission();`
- ▶ Returns permission status that is same as for `getCameraPermissionStatus()`

Showing the Camera

```
▶ const devices = useCameraDevices();  
  const device = devices.back;
```

```
  if (device == null) {  
    return <Text>No camera</Text>
```

```
  } else {  
    return (  
      <Camera  
        style={...}  
        device={device}  
        isActive={true}  
      />  
    )  
  }
```

Exercises

- ▶ Create an app showing the camera permission, and a button that allows to request the permission from the user.
- ▶ Modify the app above by adding a button to show/close the camera view once the permission is granted.

Taking a Photo

```
const camera = useRef(null);

return <>
  <Camera
    ref={camera}
    style={...}
    device={device}
    isActive={true}
    photo={true}
  />

  <Button title="Take photo" onPress={async () => {
    if (camera.current) {
      const photo = await camera.current.takePhoto({
        qualityPrioritization: 'speed',
        flash: 'auto',
        skipMetadata: true,
      });

      // Do something with the photo file...
    }
  }} />
</>
```

Getting the Captured Photo

► Show the photo

```
► {capturedPhoto && <Image  
  style={...}  
  source={{  
    uri: 'file://' + capturedPhoto.path  
  }}  
/>}
```

► Keep the photo by bringing it into the app folder

```
► const destFolder = {  
  'ios': RNFS.MainBundlePath :  
  'android': RNFS.DocumentDirectoryPath  
}[Platform.OS];  
const destFile = destFolder + '/photo.jpg';  
await RNFS.moveFile(photo.path, destFile);
```

Exercises

- ▶ Modify the app to allow taking photos and show the most recent photo.
- ▶ Add a button that allows to switch between the front and back cameras.

More Functionalities

- ▶ Check the latest documentation for more:
 - ▶ <https://www.react-native-vision-camera.com/docs/guides>
 - ▶ Capturing a video
 - ▶ More capturing options: FPS, focusing, zooming, HDR,...

Camera Roll

react-native-cameraroll Package (3rd Party)

- ▶ Provides access to the local camera roll or photo library

- ▶ Installation

- ▶ `npm install @react-native-camera-roll/camera-roll`

- ▶ Android

- ▶ Add to: `android/settings.gradle`

- ...
`include ':@react-native-camera-roll_camera-roll'`
`project(':@react-native-camera-roll_camera-roll').projectDir = new`
`File(rootProject.projectDir, '../node_modules/@react-native-camera-`
`roll/camera-roll/android')`

- ▶ Add to: `android/app/build.gradle`

- dependencies {
 ...
 implementation project(':@react-native-camera-roll_camera-roll')

- ▶ iOS: check the documentation

- ▶ Usage

- ▶ `import { CameraRoll } from "@react-native-camera-roll/camera-roll";`

Permissions: Android (1/2)

- ▶ Permission is required to read and write to the external storage, and declared in `AndroidManifest.xml` file:

```
▶ <manifest>
    ...
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    ...
</manifest>
```

- ▶ Check for permission:

```
▶ const getCheckPermissionPromise = () => {
    if (Platform.Version >= 33) {
        return Promise.all([
            PermissionsAndroid.check(
                PermissionsAndroid.PERMISSIONS.READ_MEDIA_IMAGES),
            PermissionsAndroid.check(
                PermissionsAndroid.PERMISSIONS.READ_MEDIA_VIDEO)
        ]).then( ([hasRMIP, hasRMVP]) => hasRMIP && hasRMVP );
    } else {
        return PermissionsAndroid.check(
            PermissionsAndroid.PERMISSIONS.READ_EXTERNAL_STORAGE);
    }
};
```

Permissions: Android (2/2)

► Request for permission:

```
► const getRequestPermissionPromise = async () => {  
  if (Platform.Version >= 33) {  
    const sts = await PermissionsAndroid.requestMultiple([  
      PermissionsAndroid.PERMISSIONS.READ_MEDIA_IMAGES,  
      PermissionsAndroid.PERMISSIONS.READ_MEDIA_VIDEO  
    ]);  
  
    return sts[PermissionsAndroid.PERMISSIONS.READ_MEDIA_IMAGES]  
      === PermissionsAndroid.RESULTS.GRANTED  
      && sts[PermissionsAndroid.PERMISSIONS.READ_MEDIA_VIDEO]  
      === PermissionsAndroid.RESULTS.GRANTED;  
  
  } else {  
    const sts = PermissionsAndroid.request(  
      PermissionsAndroid.PERMISSIONS.READ_EXTERNAL_STORAGE);  
    return sts === PermissionsAndroid.RESULTS.GRANTED;  
  }  
};
```

Methods

- ▶ `save(tag: string, { type: enum('photo', 'video'), album: string })`
 - ▶ Saves the photo or video to the photo library
 - ▶ On Android, the tag must be a local image or video URI, such as `"file:///sdcard/img.png"`.
 - ▶ On iOS, the tag can be any image URI (including local, remote asset-library and base64 data URIs) or a local video file URI
- ▶ `getAlbums({ assetType: string }): Promise<Array[{ title: string, count: number }]>`
 - ▶ Possible asset types: `'All', 'Videos', 'Photos'`
 - ▶ Get the list of albums
- ▶ `getPhotos({ ... }): Promise<Array[...]>`
 - ▶ A lot of options and returned information to explore more in the documentation
 - ▶ Get the list of photos

Exercise

- ▶ Write an app that asks for user permission, then shows a list of photos from the library